MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión E del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_leon_jonatan
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/leon_jonatan/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué significa que un objeto sea una instancia de una clase? Explica si dos instancias deben tener iguales valores en sus atributos.

Tu respuesta

```
Significa que el objeto sera parte o estara dentro de la clase, y 2 instancias no deben o no necesitan tener valores iguales en sus atributos.
```

Corrección. Acierta en que dos instancias no necesitan tener los mismos valores en sus atributos. Falla en decir que el objeto está dentro de la clase o forma parte de ella: una instancia es un objeto concreto que se crea a partir de la definición de la clase.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué diferencia hay entre un parámetro y un argumento de un método? Ilústrala con una llamada sencilla.

Tu respuesta

```
Un parametro es un dominio por asi decirlo donde la funcion o el objeto va a estar
funcionando, pero solo si se cumple que este dentro de ese parametro. El argumento por otro
lado es como un tipo de especificacion por asi decirlo donde se cumplen tales cosas.
Parametro:
for( i = 0;  i < 10; i++){
}

Argumento:
```

```
public casa( ) {
this.ropero = 5;
}
```

Corrección. La explicación como dominio o especificación no es correcta, y los ejemplos (un for y un constructor) no muestran la diferencia. El parámetro es la variable declarada en la cabecera, como int cantidad en cargar(int cantidad), y el argumento es el valor que se pasa en la llamada, como el 3 en cargar(3).

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué es el ámbito de una variable local? Da un ejemplo de dónde se puede usar y dónde deja de estar disponible.

Tu respuesta

Sin respuesta.

Corrección. Sin respuesta.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué guarda una variable de tipo arreglo? Explica qué ocurre al asignarla a otra variable del mismo tipo.

Tu respuesta

```
Guarda una serie de datos, como si en una caja metieramos ciertos datos y esa caja tuviera un nombre, por ejemplo si tengo una caja llamada numeros y adentro meto [5,6,4,3], esos numeros se quedan guardados en la variable numero. Puede guardar cualquier tipo de datos primitivos como int, double, String etc.
```

Corrección. Conserva una idea válida: el arreglo agrupa varios datos bajo un nombre. Pero no dice que la variable guarda una referencia, no contesta qué pasa al asignarla (las dos variables apuntan al mismo arreglo, un alias, no una copia) y String no es un tipo primitivo.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Qué hace break dentro de un case de un switch clásico? Explica qué puede ocurrir si falta.

Tu respuesta

Sin respuesta.

Corrección. Sin respuesta.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
Integer original = 7;
Integer copia = original;
int vuelta = 0;
do {
    original = original + 2;
    System.out.println(original / 2);
    vuelta++;
} while (vuelta < 3);
String texto = "E" + copia + 1;
System.out.println(texto);
System.out.println((double) (original / 2));
```

Tu respuesta

```
4
5
6
7
E8
3.0
```

Corrección. Las tres primeras líneas (4, 5, 6) son correctas. Después escribiste una línea de más y "E8": como "E" + copia + 1 se evalúa de izquierda a derecha, se concatena y da "E71", no se suma. El prefijo E sí es correcto. La última línea debía ser 6.0, porque original vale 13 y 13/2 da 6 en división entera; 3.0 solo acierta el formato double.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = null;
boolean igual = codigo.equals("R3");
System.out.println(igual);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
False
```

Corrección. Incorrecto: compila pero truena, porque se invoca equals sobre una referencia null (NullPointerException) y nunca llega al println. No imprime false.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double voltaje = 7.6;
int lectura = voltaje;
System.out.println(lectura);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
El error se debe a que se intenta igualar una variable tipo double con un entero lo que generaria el error. O si es que compilara regresaria el valor pero sin decimal.
```

Corrección. Correcto: no compila, porque se asigna un double a una variable int y Java no hace esa conversión implícita (haría falta un cast explícito).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
Integer reserva = 4;
int nueva = reserva + 3;
System.out.println(nueva);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
7
```

Corrección. La clasificación (funciona) y la salida 7 son correctas. Faltó la regla: Java desempaqueta el Integer a int para hacer la suma (unboxing).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada banda nace vacía con capacidad propia fija. El límite de seguridad común es 12 piezas. agregarPieza() suma una y consultarPiezas() responde cuántas hay. Supón que solo se llama agregarPieza cuando ambos límites lo permiten.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Cuál es final de instancia y cuál static final: capacidad o límite de seguridad?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
public class banda (){
   //Atributos
   static int bandavacia = 0;
   static int capmaxbanda = 12;
   int piezasenbanda;

   //Constructor
   class banda (){
     this.bandavacia = bandavacia;
     this.capmaxbanda = capmaxbanda;
     this.piezasenbanda = piezasenbanda;
    }

   //Metodos
   class cambio(){

    }

   class consulta(){
     if(this.piezasenbanda > 0){
          System.out.println("Hay " + this.piezasenbanda +
          " piezas en la banda");
       }
    }
}
```

Justificaciones

```
Lo primero que declare fueron los atributos, 2 static que yo asigne a los 2 datos que nos dan
que es que al inicio la banda esta vacia y el otro dato donde la capacidad maxima de la banda
es 12 y un int que correspondera a las piezas que estaran circulando por la banda. Despues
declare el constructor asignando que this.variable fuera igual a la variable original para
empezar a poder utilizar los objetos, por ultimo para los metodos no entendi a que se referia
```

```
con cambio pero para la consulta use la variable que defini para las piezas en banda por ende
podriamos consultar cuantas piezas hay en ella.
```

Corrección. Piezasenbanda como int de instancia y el límite de 12 como static son decisiones correctas y están justificadas. Falta la capacidad propia de cada banda como final de instancia, y el límite debía ser static final. bandavacia no es un atributo, es el valor inicial de piezas. El constructor se asigna cada campo a sí mismo y no recibe la capacidad. cambio quedó vacío y consulta usa un if e imprime en vez de devolver piezas. El reto no tiene respuesta: capacidad es final de instancia y LIMITE_SEGURIDAD es static final.

R4 · Código con repetición

Pregunta

Un equipo parte con carga unidades. En cada paso suma aporte. Devuelve cuántos pasos necesita para alcanzar o superar meta usando while. Si ya llegó, devuelve 0. Carga y meta están entre 0 y 10000; aporte está entre 1 y 100. No uses una fórmula de división.

```
static int pasosCarga(int carga, int aporte, int meta)
```

Ejemplos (argumentos → retorno):

5, 4, 16 → 3 · 16, 4, 16 → 0 · 20, 4, 16 → 0

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

Sin respuesta.

Corrección. Sin respuesta.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Decide si un banco de pruebas puede arrancar. Recibes el porcentaje de batería y si la tapa está cerrada. Aplica la primera regla que corresponda, en el orden de la tabla. Usa if / else-if / else, con && y ||. No imprimas ni leas del teclado.

```
static String permiso(int bateria, boolean tapaCerrada)
```

Regla o valor recibido	Texto devuelto
Batería menor que 0 o mayor que 100	"INVALIDA"
Tapa cerrada y batería al menos 60	"NORMAL"
Tapa cerrada y batería al menos 20	"AHORRO"
Cualquier otro caso	"BLOQUEADO"

Tu respuesta

```
static String permiso(int bateria, boolean tapaCerrada){
      if(bateria < 0 && bateria > 100){
         System.out.println("Invalida");
```

```
      } else if(tapaCerrada = true && bateria < 60){
         System.out.println("Normal");
      } else if(tapaCerrada = true && bateria < 20){
         System.out.println("Ahorro");
      } else{ "Bloqueado");

}
```

Corrección. La cabecera y la cadena if/else-if/else en el orden de la tabla están bien, y el else para BLOQUEADO también. Fallas de lógica: se usa = en vez de == (basta con escribir tapaCerrada) y las comparaciones están invertidas; debían ser bateria >= 60 y bateria >= 20. En los límites, la validación necesita || en lugar de &&, porque bateria < 0 && bateria > 100 nunca se cumple. Además imprime en vez de hacer return y el else está mal escrito.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.